

TaskFlow API Contract

Frontend integration reference for authentication, accounts, employees, projects, tickets, workflows, and attachments.

Version 1.0 | September 2026

General

- Production base URL: `http://<server>:8080`
- For a frontend hosted on the same IIS site, use relative URLs such as `/api/Auth/login`.
- JSON fields use `camelCase` and dates use `YYYY-MM-DD`.
- The current API does not issue a JWT and does not require a Bearer token.
- Errors return `{ "message": "Error description" }`.

Enum Values

Enum	Values
gender	0 = M, 1 = F
projectStatus	0 = Active, 1 = Completed, 2 = OnHold, 3 = Cancelled
ticketPriority	0 = Low, 1 = Medium, 2 = High
ticketStatus	0 = Pending, 1 = InProgress, 2 = InReview, 3 = NeedReview, 4 = Completed, 5 = Cancelled, 6 = Reopened, 7 = Done

JSON enum fields are sent as numbers. Route and query enum values can use enum names unless the route requires an integer.

Authentication

Sign up

POST `/api/Auth/signup`

```
{
  "fName": "Ahmad",
  "lName": "Saleh",
  "email": "ahmad@example.com",
  "phone": "0599000000",
  "gender": 0,
  "password": "Password123!",
  "confirmPassword": "Password123!",
  "acceptTerms": true
}
```

Returns `201 Created` with `{ "id": 13, "email": "ahmad@example.com", "employeeId": 13 }`. The account remains unverified until the emailed code is confirmed.

Email verification

Method	Endpoint	Body	Success
POST	/api/Auth/verify-email	{ "email": "ahmad@example.com", "code": "123456" }	200, message
POST	/api/Auth/resend-verification-code	{ "email": "ahmad@example.com" }	200, message

Login

POST /api/Auth/login

```
{  
  "email": "ahmad@example.com",  
  "password": "Password123!"  
}
```

Returns 200 OK:

```
{  
  "employeeId": 13,  
  "email": "ahmad@example.com",  
  "fName": "Ahmad",  
  "lName": "Saleh"  
}
```

Returns 401 Unauthorized for invalid credentials, an unverified email, or an inactive account.

Password recovery

Method	Endpoint	Body	Success
POST	/api/Auth/forgot-password	{ "email": "ahmad@example.com" }	200, reset code emailed
POST	/api/Auth/verify-reset-code	{ "email": "ahmad@example.com", "code": "123456" }	200, message

POST /api/Auth/reset-password

```
{  
  "email": "ahmad@example.com",  
  "code": "123456",  
  "newPassword": "NewPassword123!",  
  "confirmPassword": "NewPassword123!"  
}
```

Returns 200 OK with a message.

Accounts

Method	Endpoint	Purpose	Success
GET	/api/Account/{email}	Get account by URL-encoded email	200, account
POST	/api/Account/reactivate/{email}	Reactivate account	200, message

Account response: { "id": 13, "email": "ahmad@example.com", "employeeId": 13 }.

Update account

PUT /api/Account/{id}

```
{
  "email": "new@example.com",
  "currentPassword": "Password123!",
  "newPassword": "NewPassword123!",
  "confirmNewPassword": "NewPassword123!"
}
```

All fields are optional. Returns **200 OK** with the account response.

Deactivate account

DELETE `/api/Account/deactivate`

```
{
  "email": "ahmad@example.com",
  "currentPassword": "Password123!"
}
```

Returns **200 OK** with a message. This is a soft delete.

Employees

Employee response:

```
{
  "id": 13,
  "fName": "Ahmad",
  "lName": "Saleh",
  "phone": "0599000000",
  "gender": 0,
  "profileImageUrl": "/api/Employee/13/ProfilePhoto/file.jpg",
  "isDeleted": false
}
```

Method	Endpoint	Success
GET	<code>/api/Employee</code>	200 , employee array
GET	<code>/api/Employee/{id}</code>	200 , employee
GET	<code>/api/Employee/{id}/projects</code>	200 , project summary array
DELETE	<code>/api/Employee/{id}</code>	200 , message; soft delete
POST	<code>/api/Employee/reactivate/{id}</code>	200 , message

PUT `/api/Employee/{id}` accepts `{ "fName": "Ahmad", "lName": "Saleh", "phone": "0599000000", "gender": 0 }` and returns the updated employee.

Profile photo

POST `/api/Employee/{id}/ProfilePhoto` uses `multipart/form-data` field `file`. Maximum size is 5 MB; JPG, PNG, GIF, and WEBP are allowed. It returns the updated employee. Prefix `profileImageUrl` with the API origin when displaying it from another origin.

Projects

Project response:

```
{
  "id": 1,
  "projectName": "Website Redesign",
  "projectDescription": "Redesign the company website",
  "projectStatus": "Active",
  "startDate": "2026-09-01",
  "endDate": "2026-10-01",
  "employeeRole": "Frontend Developer",
  "projectManagerId": 2,
  "projectManagerName": "Sara Ahmad",
  "employeeCount": 4,
}
```

```
"ticketCount": 10,
"inProgressTicketCount": 3,
"needReviewTicketCount": 2,
"doneTicketCount": 5,
"progressPercentage": 50
}
```

Method	Endpoint	Success
GET	/api/Project/{id}	200, project
GET	/api/Project/Employees/{projectId}	200, employee array
GET	/api/Project/Employee/{employeeId}	200, project array
GET	/api/Project/ProjectCount/{employeeId}	200, integer
GET	/api/Project/Dashboard/{employeeId}	200, up to 3 [projectName, role] arrays
GET	/api/Project/Dashboard/{employeeId}/Projects	200, active/on-hold projects
GET	/api/Project/Employee/{employeeId}/Filter?filterStatus=Active	200, filtered projects
DELETE	/api/Project/{projectId}/{actionByEmployeeId}	204; status becomes Cancelled

Create and update project

POST /api/Project

```
{
  "projectName": "Website Redesign",
  "projectDescription": "Redesign the company website",
  "projectManagerId": 2,
  "startTime": "2026-09-01",
  "endTime": "2026-10-01"
}
```

Returns **200 OK** with the new project ID.

PUT /api/Project/Update/{projectId}/{actionByEmployeeId}

```
{
  "projectName": "Website Redesign",
  "projectDescription": "Updated description",
  "projectManagerId": 2,
  "startDate": "2026-09-01",
  "endDate": "2026-10-15"
}
```

Returns **204 No Content**. Update status with **PUT /api/Project/UpdateStatus/{projectId}/{statusNumber}**.

Project members

POST /api/Project/AddEmployee

```
{
  "projectId": 1,
  "employeeId": 3,
  "role": "Frontend Developer"
}
```

DELETE /api/Project/RemoveEmployee

```
{
  "projectId": 1,
  "employeeId": 3,
  "actionByEmployeeId": 2
}
```

Adding returns **200**; removing returns **204**.

Tickets

Ticket response:

```
{
  "ticketId": 10,
  "ticketTitle": "Build login page",
  "dueTo": "2026-09-30",
  "ticketStatus": "InProgress",
  "priority": "High",
  "description": "Create responsive login UI",
  "employeeId": 3,
  "projectId": 1,
  "projectName": "Website Redesign",
  "employeeName": "Ahmad Saleh",
  "attachments": []
}
```

Method	Endpoint	Success
GET	/api/Ticket/{id}	200, ticket
GET	/api/Ticket/Project/{projectId}	200, non-cancelled ticket array
GET	/api/Ticket/Employee/{employeeId}	200, ticket array
GET	/api/Ticket/Employee/{employeeId}/TicketCount	200, integer
GET	/api/Ticket/Employee/{employeeId}/CompletedCount	200, integer
GET	/api/Ticket/Employee/{employeeId}/InProgressCount	200, integer
GET	/api/Ticket/Employee/{employeeId}/NeedReviewCount	200, integer
GET	/api/Ticket/{ticketId}/History	200, history array
GET	/api/Ticket/{ticketId}/Attachments	200, attachment array
DELETE	/api/Ticket/{id}	204; status becomes Cancelled

Create and update ticket

POST /api/Ticket

```
{
  "ticketTitle": "Build login page",
  "dueTo": "2026-09-30",
  "priority": 2,
  "description": "Create responsive login UI",
  "attachmentURL": null,
  "employeeId": 3,
  "ticketCreatedBy": 2,
  "projectId": 1
}
```

The assignee must belong to the project. Returns 200 OK with the new ticket ID.

PUT /api/Ticket/{id} accepts { "ticketTitle": "Updated title", "dueTo": "2026-10-02", "description": "Updated description", "employeeId": 3 } and returns 204.

Status, priority, and workflow

Method	Endpoint	Body	Success
PUT	/api/Ticket/Status/{ticketId}/{status}	none	204
PUT	/api/Ticket/Priority/{ticketId}/{priority}	none	204
PUT	/api/Ticket/{ticketId}/SubmitReview	action body	204
PUT	/api/Ticket/{ticketId}/Review/Approve	action body	204
PUT	/api/Ticket/{ticketId}/Review/RequestChanges	action body	204
POST	/api/Ticket/{ticketId}/Comments	action body	204

Action body: { "actionByEmployeeId": 3, "note": "Work is ready for review." }.

Reassign with PUT /api/Ticket/{ticketId}/Review/Reassign:

```
{
  "actionByEmployeeId": 2,
  "toEmployeeId": 4,
  "note": "Please handle the requested changes."
}
```

Ticket history

```
{
  "id": 1,
  "ticketId": 10,
  "action": "StatusChanged",
  "oldValue": "InProgress",
  "newValue": "InReview",
  "note": "Work is ready for review.",
  "actionByEmployeeId": 3,
  "actionByEmployeeName": "Ahmad Saleh",
  "fromEmployeeId": null,
  "fromEmployeeName": null,
  "toEmployeeId": null,
  "toEmployeeName": null,
  "modifiedAt": "2026-09-09T12:30:00"
}
```

Ticket Attachments

Operation	Endpoint	Form field	Limits
Upload one	POST /api/Ticket/Attachments/upload/Ticket/{ticketId}	file	10 MB
Upload multiple	POST /api/Ticket/Attachments/upload/Ticket/{ticketId}/Multiple	repeated files	5 files, 10 MB each

Upload response items contain `fileName`, `originalFileName`, `contentType`, `sizeInBytes`, and `url`.

Attachment response:

```
{
  "id": 7,
  "ticketId": 10,
  "originalFileName": "requirements.pdf",
  "storedFileName": "generated-file-name.pdf",
  "contentType": "application/pdf",
  "sizeInBytes": 120000,
  "url": "/api/Ticket/Attachments/download-file/generated-file-name.pdf"
}
```

Use GET {apiOrigin}{url} to download or display the attachment.

Complete Endpoint Index

This index lists every active route declared by the current backend. The routes in the main tables are the canonical routes the frontend should use.

System and authentication

Method	Endpoint	Purpose	Environment
GET	/	Health response: Server is working!	All
GET	/openapi/v1.json	OpenAPI document	Development only
GET	/scalar	Interactive API reference	Development only
POST	/api/Auth/signup	Create employee/account and send verification code	All
POST	/api/Auth/verify-email	Verify signup email	All
POST	/api/Auth/resend-verification-code	Send a new verification code	All
POST	/api/Auth/login	Sign in	All
POST	/api/Auth/forgot-password	Send password reset code	All
POST	/api/Auth/verify-reset-code	Validate password reset code	All
POST	/api/Auth/reset-password	Set a new password	All

Account and employee

Method	Endpoint	Purpose	Response
GET	/api/Account/{email}	Get account by email	Account
POST	/api/Account/reactivate/{email}	Reactivate account	Message
PUT	/api/Account/{id}	Update email/password	Account
DELETE	/api/Account/deactivate	Deactivate account	Message
GET	/api/Employee	Get employees	Employee array
GET	/api/Employee/{id}	Get employee	Employee
GET	/api/Employee/{id}/projects	Get employee project summaries	Project array
PUT	/api/Employee/{id}	Update employee	Employee
POST	/api/Employee/{id}/ProfilePhoto	Upload profile photo	Employee
GET	/api/Employee/{id}/ProfilePhoto/{fileName}	Read profile photo	File
DELETE	/api/Employee/{id}	Soft-delete employee	Message
POST	/api/Employee/reactivate/{id}	Reactivate employee	Message

Project

Method	Endpoint	Purpose	Response
DELETE	/api/Project/{id}/{empId}	Cancel project	204
GET	/api/Project/ProjectCount/{employeeId}	Count employee projects	Integer
POST	/api/Project	Create project	Project ID
PUT	/api/Project/UpdateStatus/{id}/{statusNumber}	Change project status	204
GET	/api/Project/{id}	Get project	Project
GET	/api/Project/Employees/{projectId}	Get project employees	Employee array
PUT	/api/Project/Update/{id}/{empId}	Update project	204
GET	/api/Project/Employee/{employeeId}	Get employee projects	Project array
POST	/api/Project/AddEmployee	Add project member	Submitted body
DELETE	/api/Project/RemoveEmployee	Remove project member	204
GET	/api/Project/Dashboard/{employeeId}	Get latest three projects	String arrays
GET	/api/Project/Dashboard/{employeeId}/Projects	Get dashboard projects	Project array
GET	/api/Project/Employee/{employeeId}/Filter?filterStatus={status}	Filter employee projects	Project array

Ticket

Method	Endpoint	Purpose	Response
GET	/api/Ticket/Project/{projectId}	Get project tickets	Ticket array
GET	/api/Ticket/Employee/{employeeId}	Get employee tickets	Ticket array
GET	/api/Ticket/Employee/{employeeId}/TicketCount	Total employee tickets	Integer
GET	/api/Ticket/{id}	Get ticket	Ticket
GET	/api/Ticket/{ticketId}/History	Get ticket history/comments	History array
GET	/api/Ticket/{ticketId}/Attachments	Get attachment metadata	Attachment array
POST	/api/Ticket	Create ticket	Ticket ID
PUT	/api/Ticket/{id}	Update ticket	204
DELETE	/api/Ticket/{id}	Cancel ticket	204
PUT	/api/Ticket/Status/{ticketId}/{status}	Change status	204
PUT	/api/Ticket/Priority/{ticketId}/{priority}	Change priority	204
PUT	/api/Ticket/{ticketId}/SubmitReview	Submit for review	204
PUT	/api/Ticket/{ticketId}/Review/Approve	Approve reviewed ticket	204
PUT	/api/Ticket/{ticketId}/Review/RequestChanges	Request changes	204
PUT	/api/Ticket/{ticketId}/Review/Reassign	Reassign ticket	204
POST	/api/Ticket/{ticketId}/Comments	Add history comment	204
GET	/api/Ticket/Employee/{employeeId}/CompletedCount	Count completed tickets	Integer
GET	/api/Ticket/Employee/{employeeId}/InProgressCount	Count in-progress tickets	Integer
GET	/api/Ticket/Employee/{employeeId}/NeedReviewCount	Count tickets needing review	Integer
POST	/api/Ticket/Attachments/upload/Ticket/{ticketId}	Upload one attachment	Upload metadata
POST	/api/Ticket/Attachments/upload/Ticket/{ticketId}/Multiple	Upload attachments	Upload metadata array
GET	/api/Ticket/Attachments/download-file/{fileName}	Download stored attachment	File
GET	/api/Ticket/Attachments/download/{URL}	Legacy file-path download	File

Duplicate route aliases

The code also declares these aliases. They reach the same actions, but the frontend should use the canonical `/api/...` routes above.

Method	Alias	Canonical route	Note
DELETE	/api/Project/Delete/{id}/{empId}	/api/Project/{id}/{empId}	Legacy alias

Method	Alias	Canonical route	Note
GET	/api/Project/employeeId%20=%20{employeeId}	/api/Project/Employee/{employeeId}	Legacy alias with spaces
GET	/api/Project/employeeId%20=%20{employeeId}/Project	/api/Project/Employee/{employeeId}/Filter	Legacy alias with spaces
GET	/Project/{projectId}	/api/Ticket/Project/{projectId}	Root-level alias
GET	/Employee/{employeeId}	/api/Ticket/Employee/{employeeId}	Root-level alias
GET	/Employee/{employeeId}/TicketCount	/api/Ticket/Employee/{employeeId}/TicketCount	Root-level alias
PUT	/Status/{ticketId}/{status}	/api/Ticket/Status/{ticketId}/{status}	Root-level alias
PUT	/Priority/{ticketId}/{priority}	/api/Ticket/Priority/{ticketId}/{priority}	Root-level alias
GET	/Employee/{employeeId}/CompletedCount	/api/Ticket/Employee/{employeeId}/CompletedCount	Root-level alias
GET	/Employee/{employeeId}/InProgressCount	/api/Ticket/Employee/{employeeId}/InProgressCount	Root-level alias
GET	/Employee/{employeeId}/NeedReviewCount	/api/Ticket/Employee/{employeeId}/NeedReviewCount	Root-level alias

The standalone `Attachment` class contains `/api/Attachment/upload`, `/api/Attachment/upload/multiple`, and `/api/Attachment/download/{fileName}` in source code. Its class name does not follow MVC controller discovery naming, so it is not part of the active API contract. Use the ticket attachment endpoints instead.

Common Status Codes

Status	Meaning
200	Succeeded and returned data or a message
201	Account created
204	Succeeded with no response body
400	Invalid request or business rule failed
401	Invalid login or employee is not allowed to perform the action
404	Resource not found
409	Duplicate or conflicting data
500	Unexpected server or configuration error